

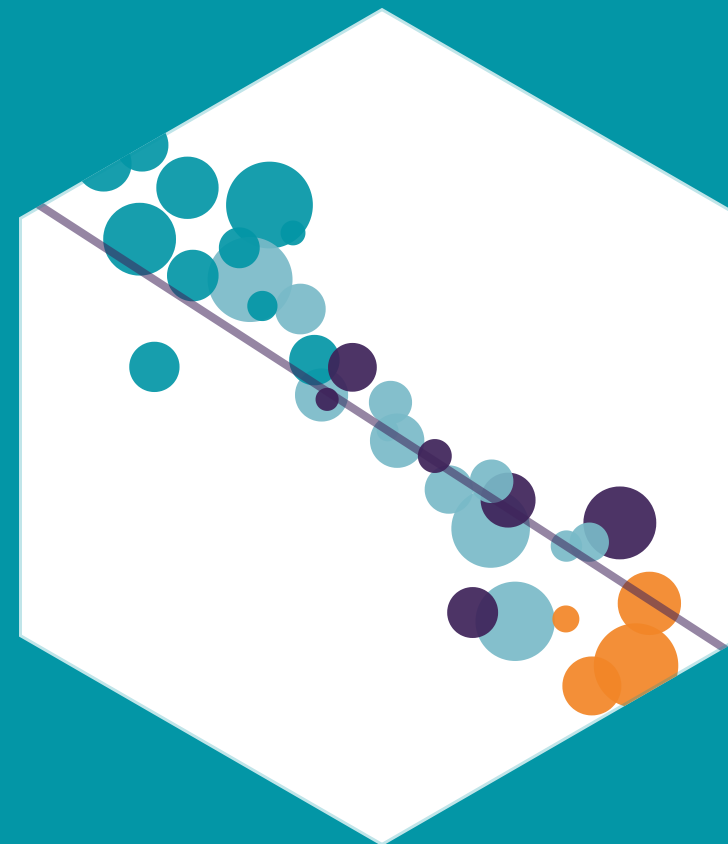
ECONOMETRICS

TA SESSION 1

Barcelona School of Economics

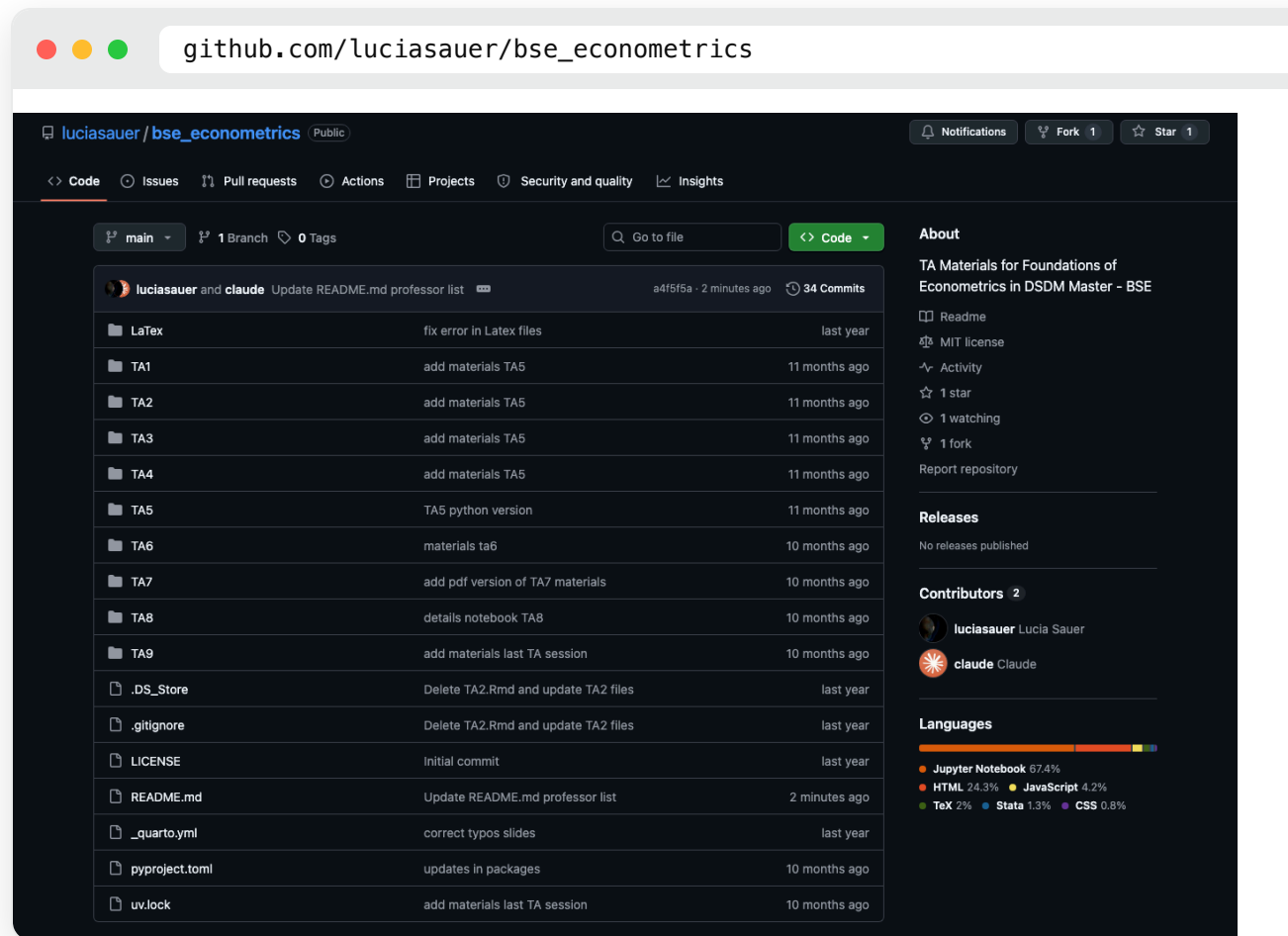
Lucia Sauer

✉ lucia.sauer@bse.eu



TA Materials

All the material for this course — slides, code, and some LaTeX utilities for referencing assignments — will be hosted in the [GitHub repository](https://github.com/luciasauer/bse_econometrics).



```
git clone HTTPS
```

Install uv

```
uv sync
```

Pull the repository and get the updated materials for each session.

Overview

- Matrix Algebra with Python and R
- Derivatives with vectors
- Histograms and KDE
- Quick LaTeX guide

Why These Topics?

Matrix Algebra

- Compact notation for regression & multivariate models
- Solve systems efficiently with Python

Derivatives with Vectors

- Gradients for optimization & ML
- Key for regression, loss functions

Histograms & KDE

- Explore data shape: skewness, modality, outliers
- Nonparametric distribution estimation & model checks

System of equations in Matrix Form

Consider the following system of equations:

$$\begin{cases} 2x_1 + 3x_2 + 10x_3 = 5 \\ 4x_1 + x_2 + 12x_3 = 6 \\ 7x_1 + 2x_2 + x_3 = 10 \end{cases}$$

Exercises

- How can we write this system in matrix form?
- Solve it using Python.

Matrix Operations using Python

```
1 import numpy as np
2
3 #Define matrix A and vector b
4 A = np.array([[2, 3, 10],
5               [4 , 1, 12],
6               [7 , 2, 1]])
7 b = np.array([5 , 6, 10])
8
9 #Inverse A
10 A_inverse = np.linalg.inv(A)
11
12 x = np.dot(A_inverse, b)
13 print('Solution: ', x)
```

Solution: [1.21078431 0.74509804 0.03431373]

Matrix Operations using R

```
1 # Define matrix A and vector b
2 A <- matrix(c(2, 3, 10,
3              4, 1, 12,
4              7, 2, 1),
5             nrow = 3, byrow = TRUE)
6
7 b <- c(5, 6, 10)
8
9 # Inverse A
10 A_inverse <- solve(A)
11 x <- A_inverse %*% b
12
13 # Solve Ax = b
14 x <- solve(A, b)
15 print(x)
```

Derivatives of a vector

Gradient with respect to a vector

- In single-variable calculus:

$\frac{d}{dx}f(x)$ is the slope of a scalar function of one variable.

- In multivariable calculus:

If $f(z_1, z_2)$, we can group the variables into a vector:

$$z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}$$

then the derivative with respect to a vector is the **gradient**:

$$\nabla_z f(z) = \begin{bmatrix} \frac{\partial f}{\partial z_1} \\ \frac{\partial f}{\partial z_2} \end{bmatrix}$$

👉 Take the partial derivative with respect to each coordinate, then stack them in a column vector.

Consider:

$$a = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} \quad z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

Exercises

Verify, by showing all the relevant steps, that:

$$(i) \quad \frac{\partial z' a}{\partial a} = z$$

$$(ii) \quad \frac{\partial a' z}{\partial z} = a$$

$$(iii) \quad \frac{\partial z' B z}{\partial z} = 2Bz \quad \text{since } B \text{ is symmetric}$$

Histograms & Kernel Density Estimation

Density Estimation - Introduction

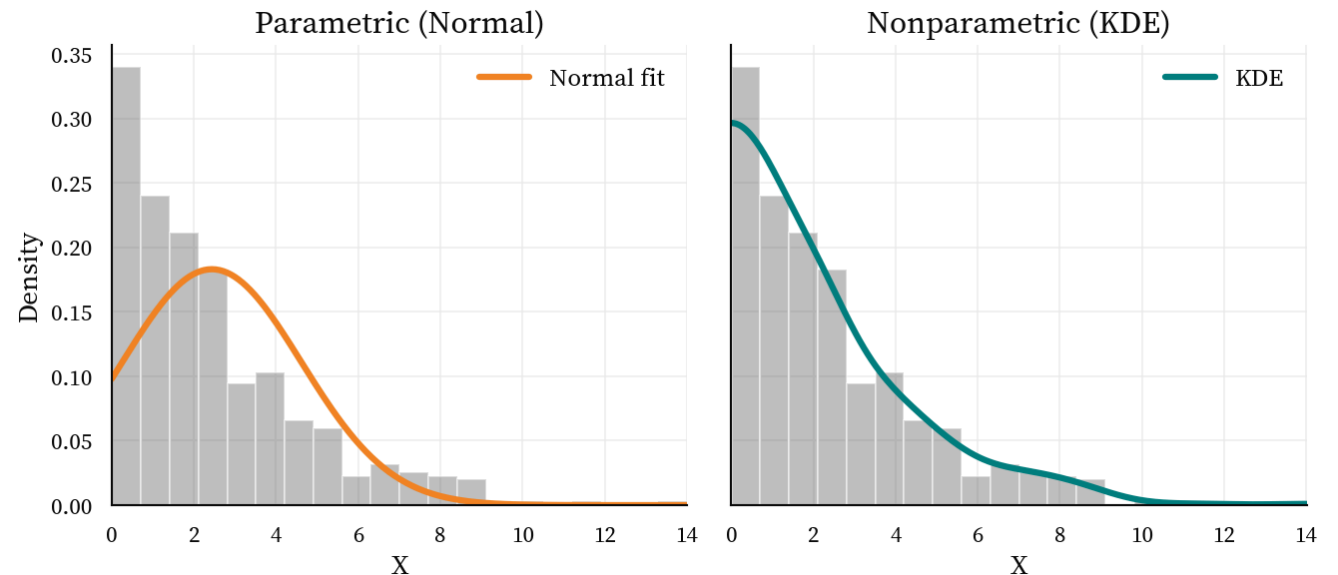
Given some observations from some variable X , we would like to obtain an estimate of its density.

Parametric vs Nonparametric

- **Parametric:** choose a density family (normal, exponential, uniform...) and estimate its parameters from data.

Problem: what if X doesn't follow that family?

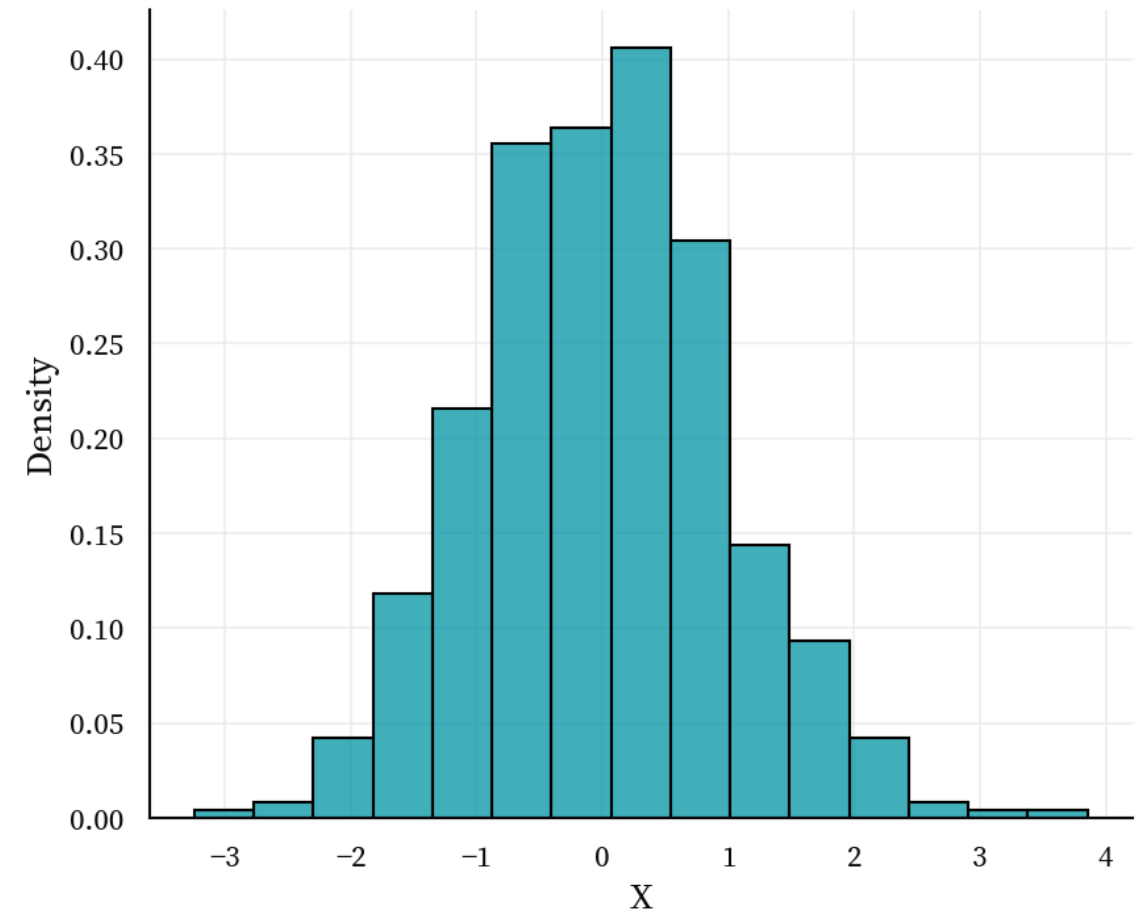
- **Nonparametric** (histograms, KDE) estimates the distribution without strong assumptions.



Histogram

Key idea: A histogram is a graphical tool to visualize the distribution of a dataset.

- A **nonparametric method** to estimate the probability density function (PDF)
- Groups data into **intervals (bins)**
- Plots the **frequency** of data points in each bin



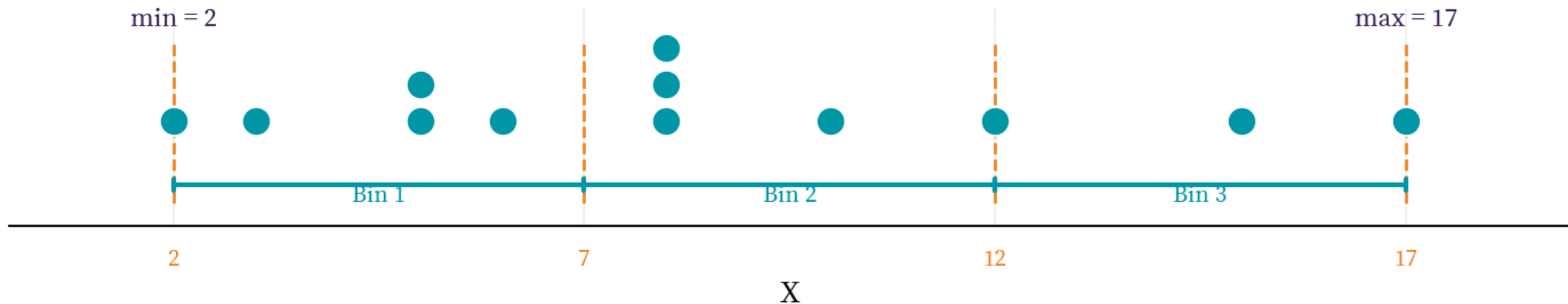
Concept of Density

Let's differentiate between frequency histograms and density histograms, using a dataset with 12 observations: [2, 3, 5, 5, 6, 8, 8, 8, 10, 12, 15, 17]

1 — 2 — 3 BINWIDTH

If we fix the number of bins ex-ante, we must derive the binwidth:

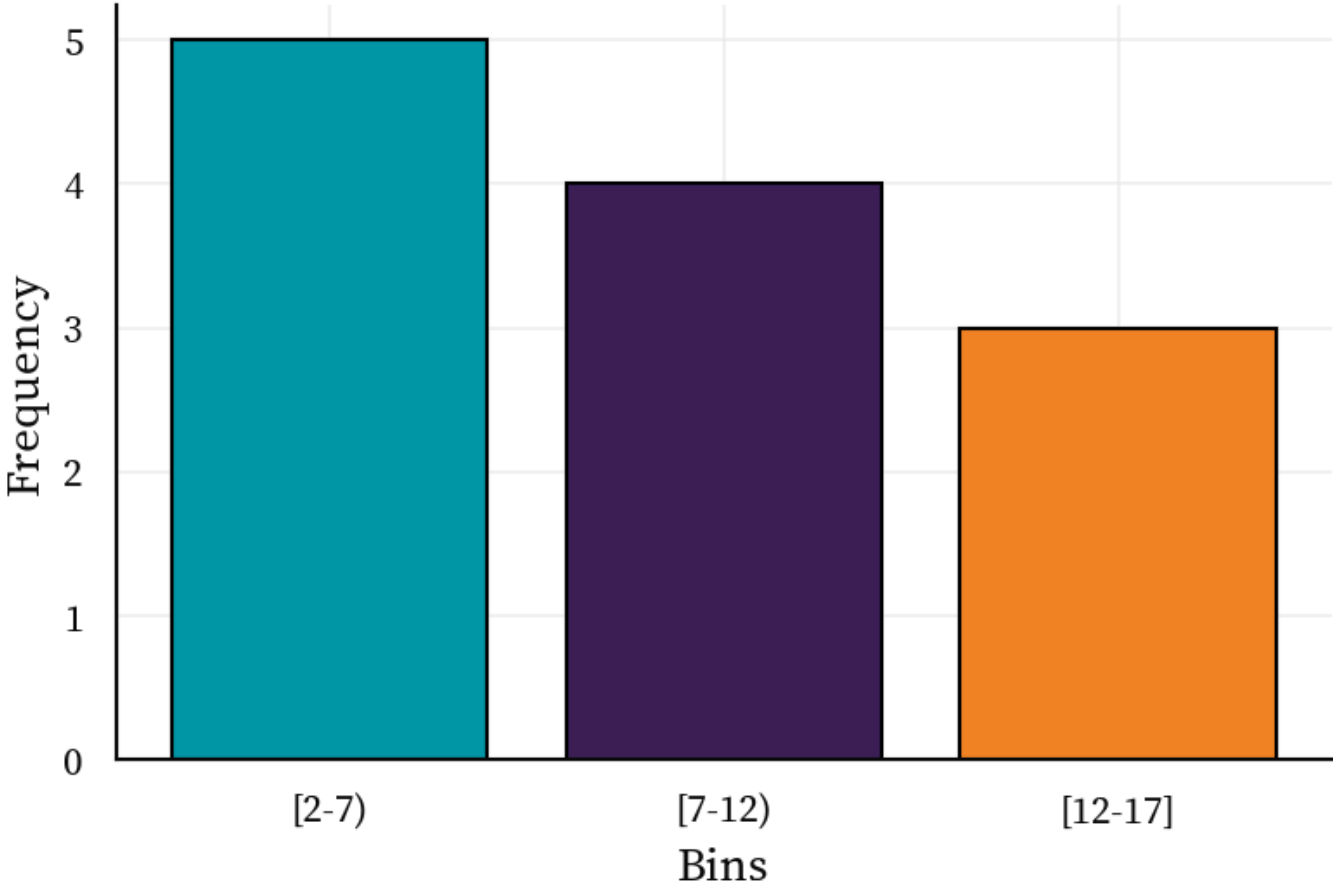
$$n_{bins} = 3 \Rightarrow \text{binwidth} = h = \frac{\max(x) - \min(x)}{n_{bins}} = \frac{17 - 2}{3} = 5$$



Concept of Density

1 — 2 — 3 BIN LIMITS

Bins	Frequency
[2-7)	5
[7-12)	4
[12-17]	3



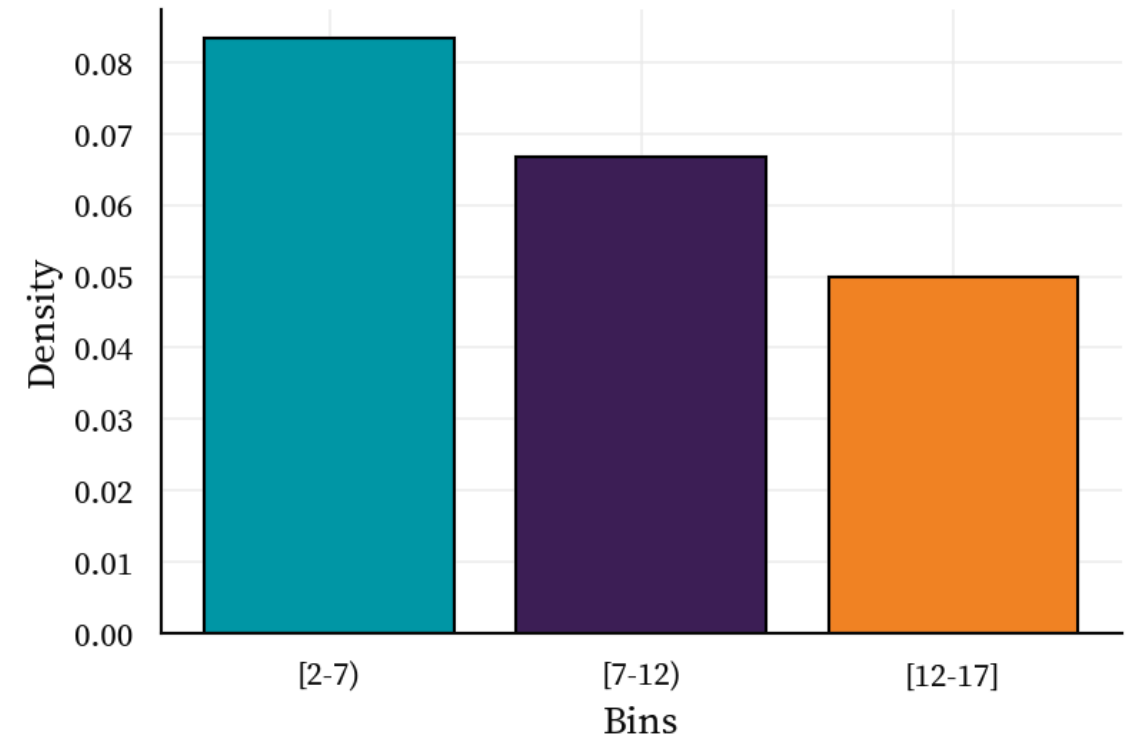
Concept of Density

1 — 2 — 3 DENSITY

Bins	Frequency	Width	Density
[2-7)	5	5	$5/(12 * 5) = 0.0833$
[7-12)	4	5	$4/(12 * 5) = 0.0667$
[12-17]	3	5	$3/(12 * 5) = 0.05$

Mathematically, for bin width h and number of observations n :

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n \mathbf{1}\{x_i \in \text{bin}(x)\}$$



Density $\neq$ Probability

Exam question

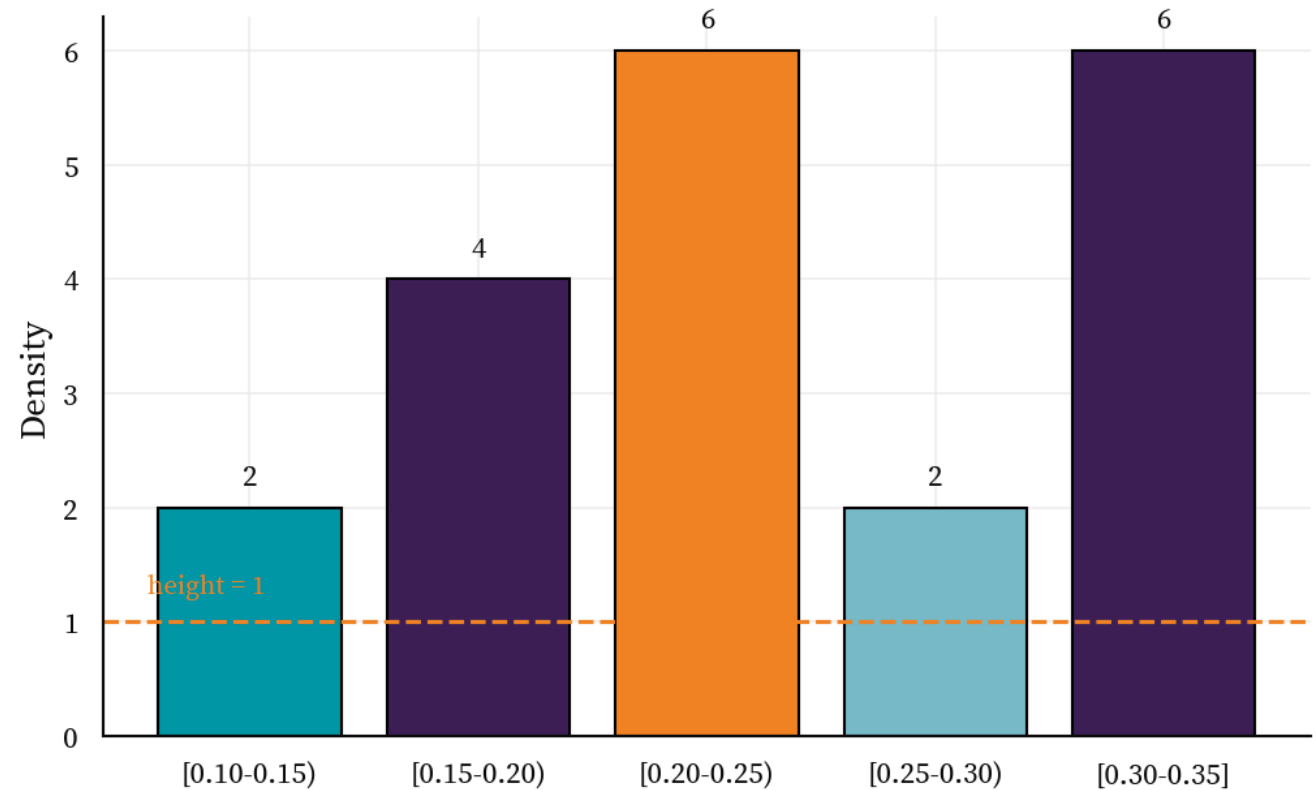
Could the range of values of the vertical axis in a histogram be greater than 1? Justify your answer.

Imagine the following data,
narrower range (0.10 to 0.35):

[0.10, 0.15, 0.15, 0.20, 0.20,
0.20, 0.25, 0.30, 0.30, 0.35]

We set 5 bins ($h = 0.25/5 = 0.05$),
usual estimator:

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n \mathbf{1}_{\{x_i \in \text{bin}(x)\}}, \quad nh = 0.5$$



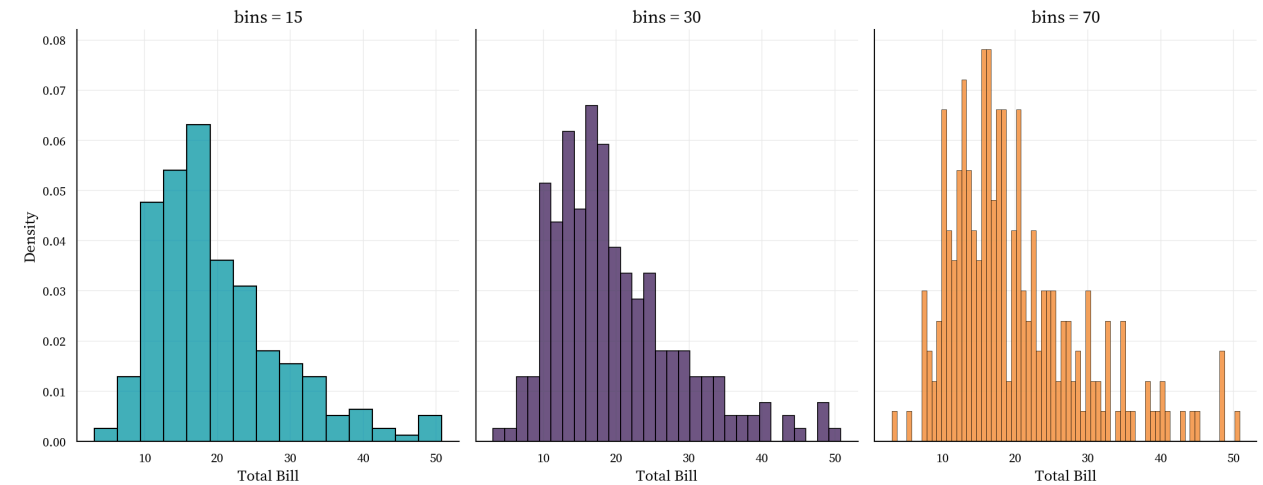
Coding Example in Python

Important parameters `bins`, `stat`, `binwidth`, `binrange`

CODE

```
1 import seaborn as sns
2 x = sns.load_dataset("tips")["total_bill"]
3 bins = [15, 30, 70]
4 for bin in bins:
5     sns.histplot(x, bins=bin, stat='density', binwidth=None, binrange=None)
```

OUTPUT



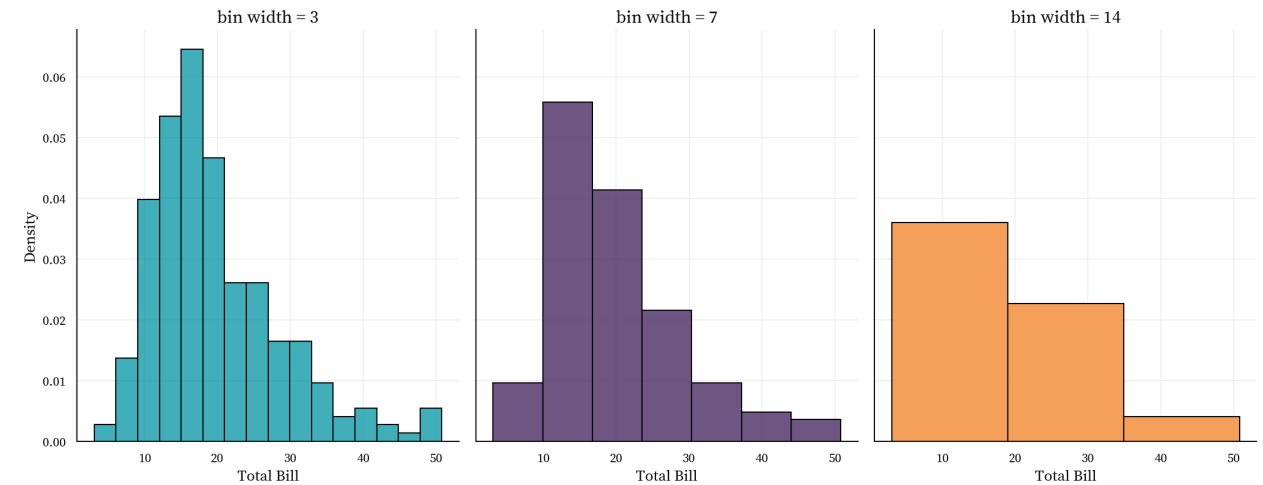
Coding Example in Python

Important parameters `bins`, `stat`, `binwidth`, `binrange`

CODE

```
1 import seaborn as sns
2 x = sns.load_dataset("tips")["total_bill"]
3 bin_widths = [3, 7, 14]
4 for bin_w in bin_widths:
5     sns.histplot(x, bins=None, stat='density', binwidth=bin_w, binrange=None)
```

OUTPUT



Coding Example in R

Important parameters: `bins`, `aes(y = ..stat..)`, `binwidth`, `xlim()`

```
1 library(ggplot2)
2 library(readr)
3
4 tips <- read_csv("https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv")
5 x <- tips$total_bill
6
7 bins <- c(15, 30, 70)
8 for (bin in bins) {
9   p <- ggplot(data = data.frame(x = x), aes(x = x)) +
10     geom_histogram(aes(y = ..density..), bins = bin, color = "black", fill = "skyblue")
11   print(p)
12 }
```

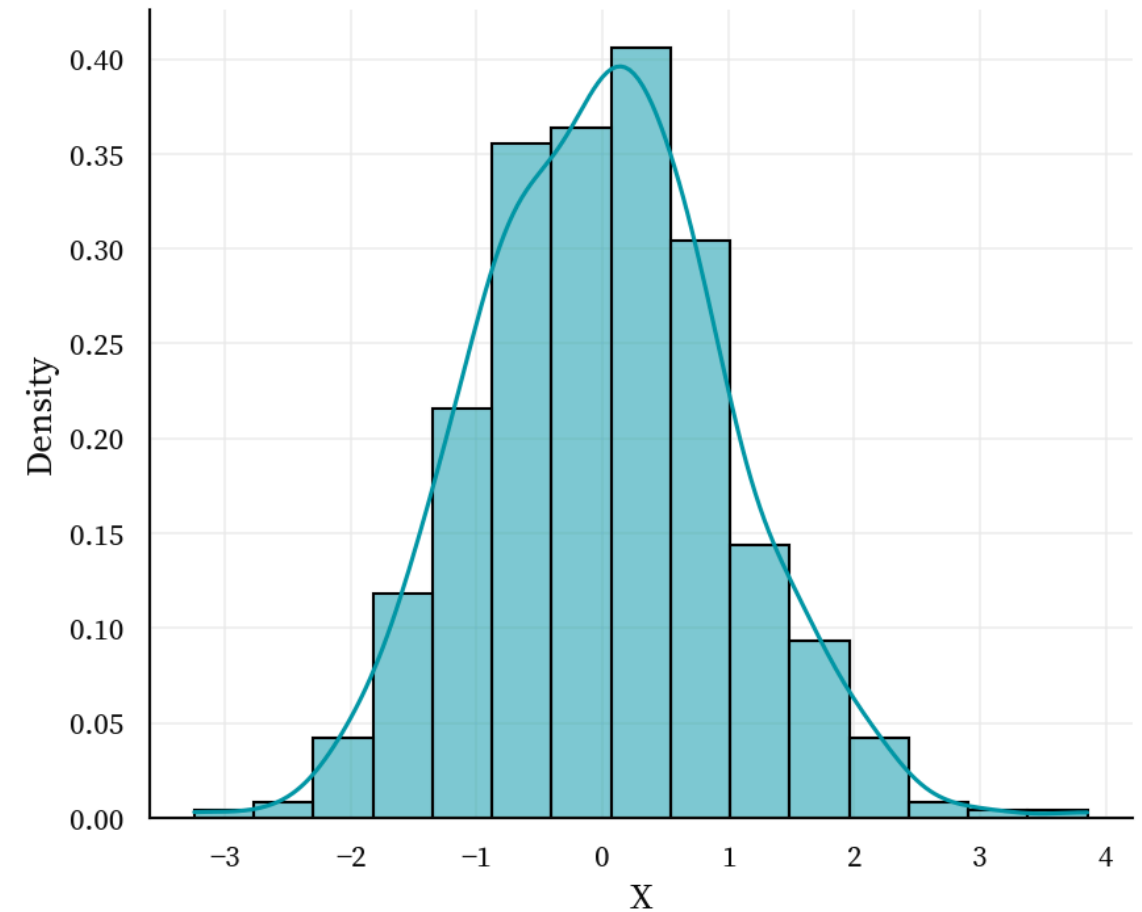
Kernel Density Estimation

Key idea: Smooth, nonparametric estimate of the probability density function

- Instead of counting frequencies in bins, KDE places a smooth kernel function (usually Gaussian) centered at each data point, and then averages them.

The limitation KDE fixes

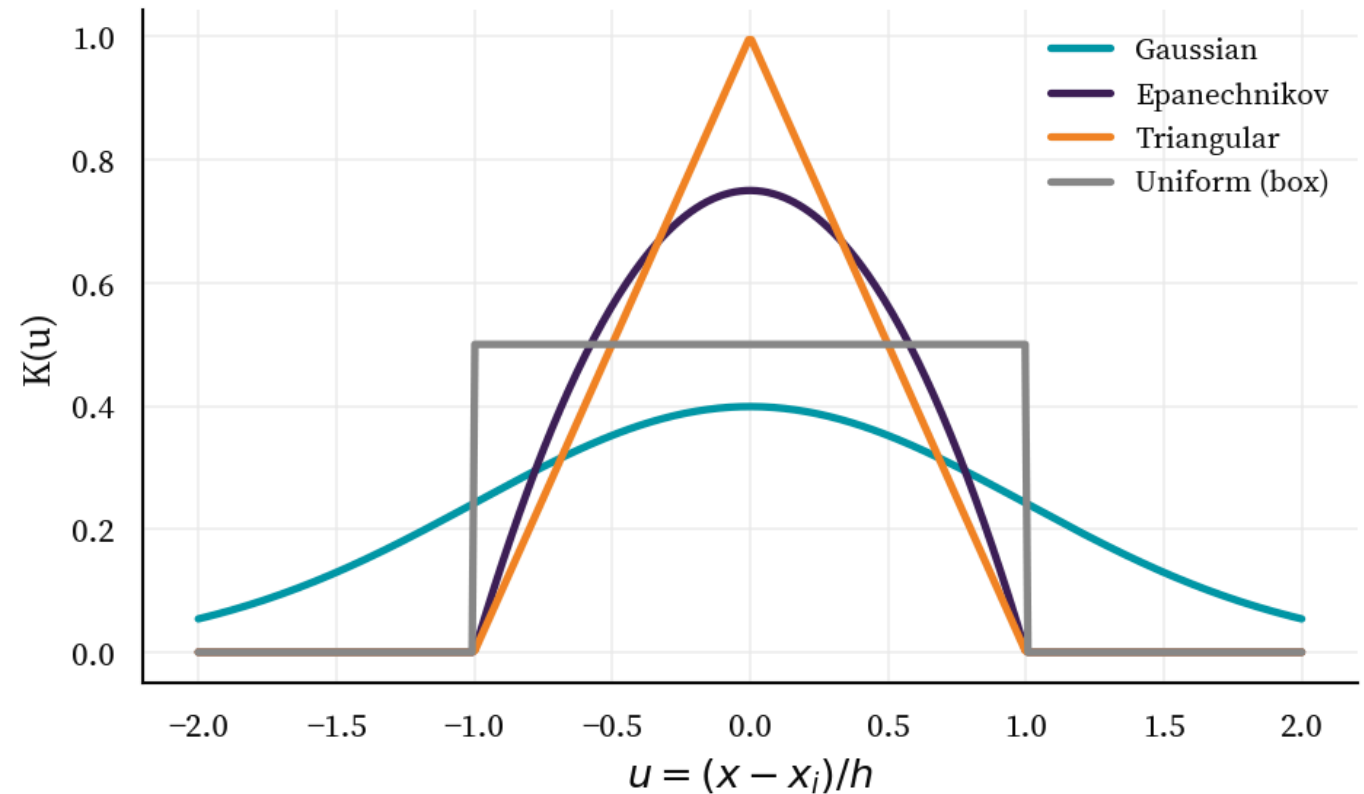
A histogram is **discrete**: it jumps abruptly at every bin edge. KDE is **continuous by construction** — no bins, no edges, just a smooth curve.



The Kernel Function

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

- n is the number of observations
- h is the bandwidth (controls how much each observation's influence spreads)
- K is the **kernel** — any symmetric function integrating to 1. Gaussian is the most common choice, but not the only one:

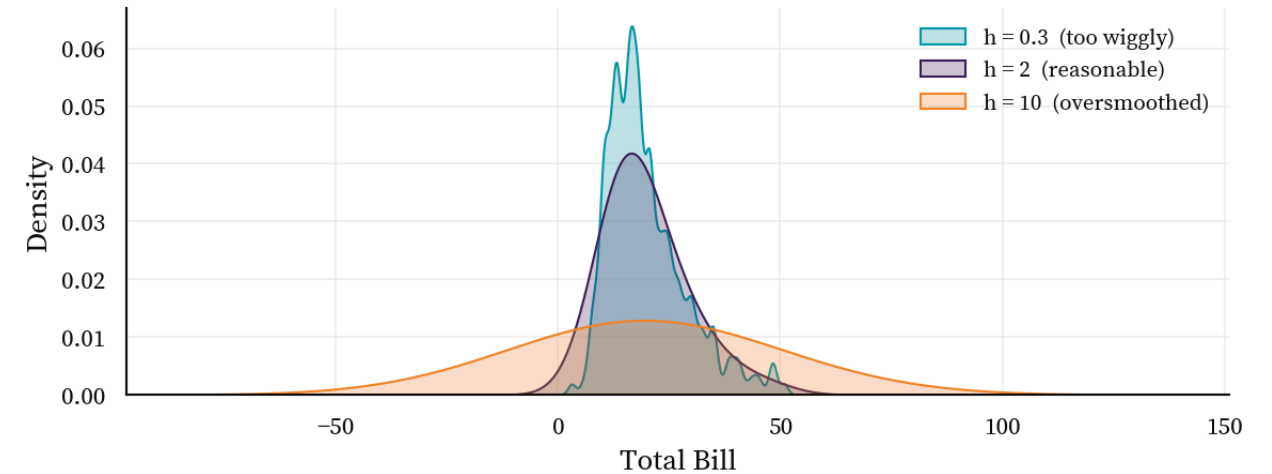


Coding Example in Python

CODE

```
1 import seaborn as sns
2 x = sns.load_dataset('tips')['total_bill']
3 for bw in [0.3, 2, 10]:
4     sns.kdeplot(x, bw_adjust=bw, label=f"Bandwidth = {bw}")
```

OUTPUT



- **Small h** : very wiggly estimate (low bias, high variance).
- **Large h** : oversmoothed estimate (high bias, low variance).
- Same bias-variance trade-off as the **binwidth** h in a histogram — just for a smooth curve instead of bars.

Coding example in R

```
1 library(ggplot2)
2 library(readr)
3
4 tips <- read_csv("https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv")
5 x <- tips$total_bill
6
7 for (bw in c(0.3, 1, 2)) {
8   print(ggplot(data.frame(x = x), aes(x = x)) +
9         geom_density(adjust = bw, color = "blue") +
10        ggtitle(paste("Bandwidth =", bw)))
11 }
```

Quick LaTeX Guide

What is LaTeX (and why use it)?

LaTeX is a **typesetting system**, not a word processor: you write plain-text markup, and it *compiles* into a professional PDF — like code, but for documents.

Why for assignments

- Clean, consistent math notation
- Regression tables & figures exported **directly** from Python/Stata — no manual formatting
- The same source always reproduces the same PDF



Barcelona School of Economics

Assignment

Econometrics

Lucia Sauer

Key Commands: Structuring a Document

```
1 \section{Introduction}
2 This is the introduction to the assignment.
3
4 \subsection{Motivation}
5 Here we explain why this analysis matters.
6
7 \subsection{Data}
8 We describe the dataset used.
9
10 \section{Results}
11 We now present our findings.
```

- `\section{}` numbers top-level headings; `\subsection{}` nests one level in
- Numbering, spacing, and the table of contents update **automatically**
- No manual “1.2” typing, no forgetting to renumber after edits

1 Introduction

This is the introduction to the assignment.

1.1 Motivation

Here we explain why this analysis matters.

1.2 Data

We describe the dataset used.

2 Results

We now present our findings.

Key Commands: Math Notation

```
1 \begin{equation}
2 y_i = \beta_0 + \beta_1 x_i + \varepsilon_i
3 \end{equation}
4
5 Matrices use the bmatrix environment:
6 \[
7 X =
8 \begin{bmatrix}
9 1 & x_{11} & x_{12} \\
10 1 & x_{21} & x_{22} \\
11 \vdots & \vdots & \vdots \\
12 1 & x_{n1} & x_{n2}
13 \end{bmatrix}, \quad \beta =
14 \begin{bmatrix}
15 \beta_0 \\
16 \beta_1 \\
17 \beta_2
18 \end{bmatrix}
19 \[
20 \hat{\beta} = (X'X)^{-1} X'Y
21 \]
```

- `equation` numbers the line; `\[...\]` (or `$...$`) doesn't
- rows separated by `\\`, entries by `&`

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i \quad (1)$$

Matrices are displayed with the `bmatrix` environment:

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} \\ 1 & x_{21} & x_{22} \\ \vdots & \vdots & \vdots \\ 1 & x_{n1} & x_{n2} \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

Solving for the OLS estimator:

$$\hat{\beta} = (X'X)^{-1}X'Y$$

From Python to a Rendered Table

```
1 import statsmodels.formula.api as smf
2 from stargazer.stargazer import Stargazer
3
4 m1 = smf.ols("tip ~ total_bill", tips).fit()
5 m2 = smf.ols("tip ~ total_bill + size", tips).fit()
6
7 sg = Stargazer([m1, m2])
8 sg.rename_covariates({"total_bill": "Total Bill", "size": "Party Size"})
9
10 with open("regression.tex", "w") as f:
11     f.write(sg.render_latex())
```

Then in LaTeX you just call the .tex file

```
1 \input{regression.tex}
```

Table 1: Effect of Bill Size on Tip Amount		
	<i>Dependent variable: tip</i>	
	(1)	(2)
Intercept	0.920*** (0.160)	0.669*** (0.194)
Party Size		0.193** (0.085)
Total Bill	0.105*** (0.007)	0.093*** (0.009)
Observations	244	244
R^2	0.457	0.468
Adjusted R^2	0.454	0.463
Residual Std. Error	1.022 (df=242)	1.014 (df=241)
F Statistic	203.358*** (df=1; 242)	105.948*** (df=2; 241)
<i>Note:</i> *p<0.1; **p<0.05; ***p<0.01		

Working in Overleaf

Free plan

- Nothing to install – compiles in the browser
- Real-time collaborative editing
- **Free tier: only 2 named editors** per project

Beyond 2 editors: link sharing

Share → **Turn on link sharing** → set the link to **Can edit**.
Anyone with that link can now edit – no named-collaborator limit applies.

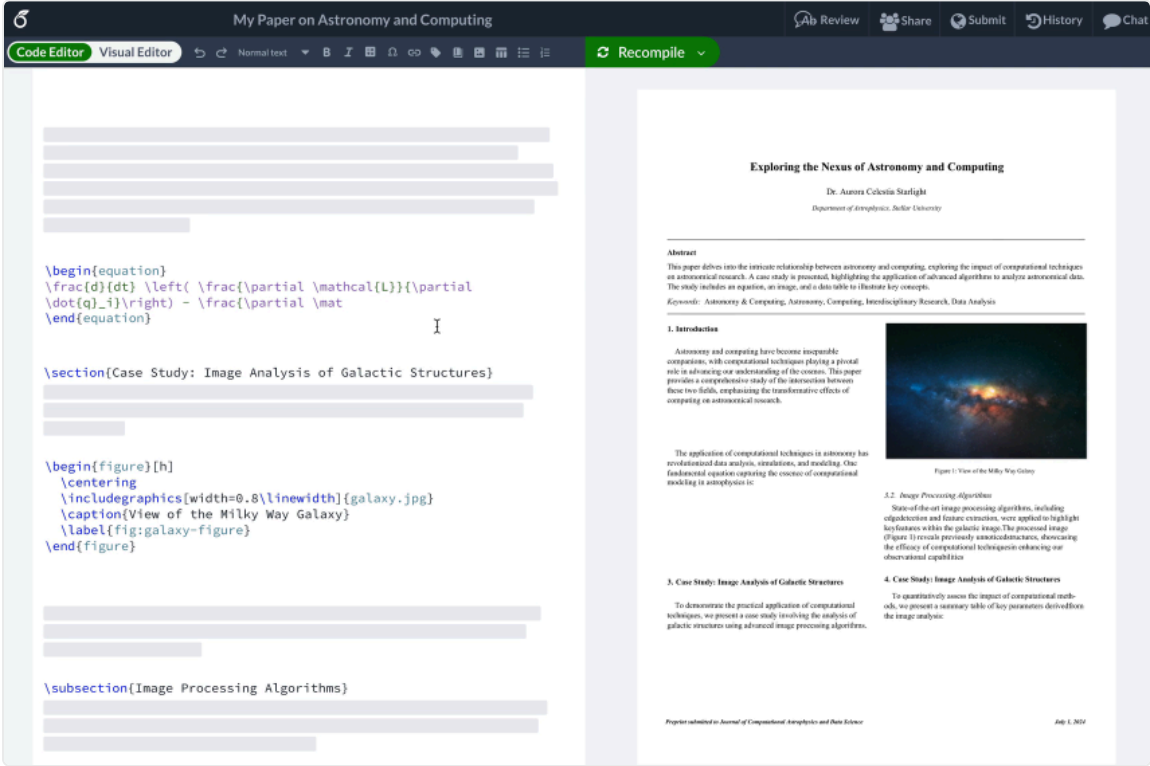
Share “Assignment 1 · Econometrics”

Invite by email 2 / 2 editors

or

 Turn on link sharing 

Can edit overleaf.com/2847...



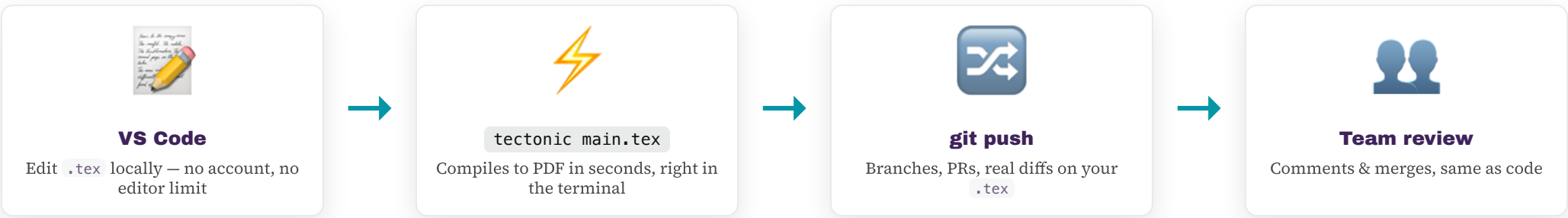
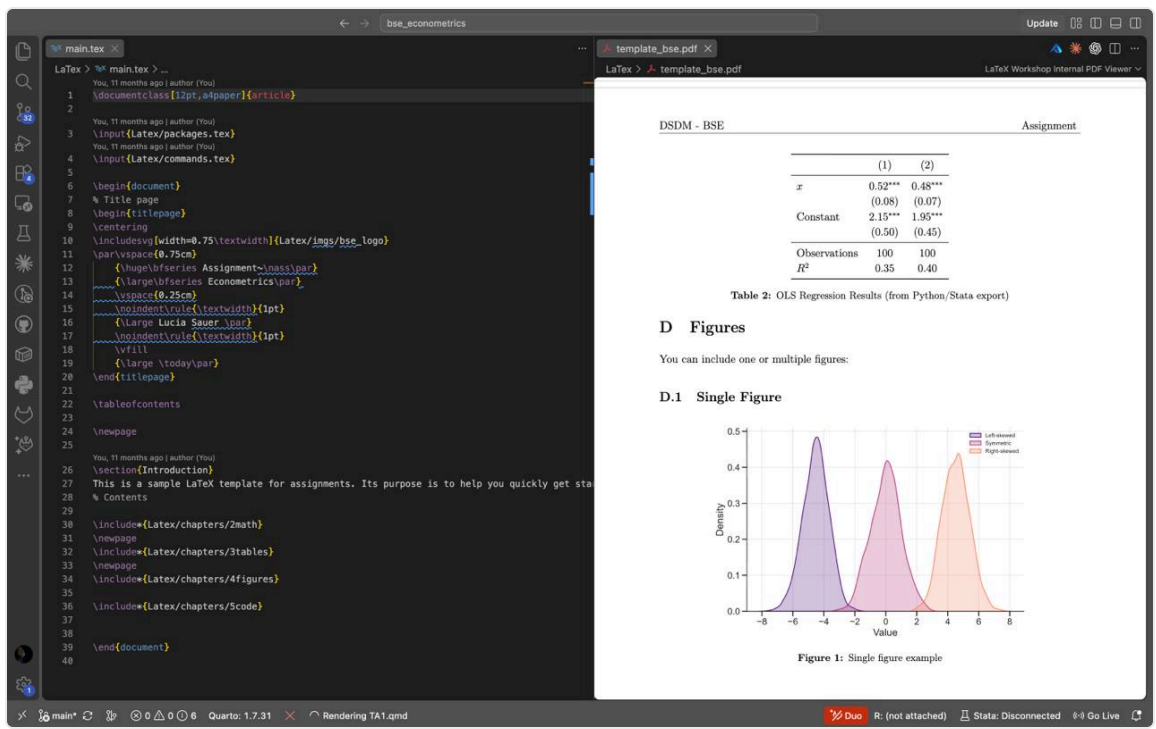
A Wilder Option: Tectonic + GitHub in VS Code

Why bother?

Overleaf's free tier caps you at 2 editors. Git doesn't cap anything — and you get real **pull requests, diffs, and code review** on your `.tex` source, the same workflow you already use for code.

Compile straight from the terminal, as many times as you want, in seconds:

```
tectonic main.tex
```



Thanks!